

PROGETTAZIONE DI SISTEMI DIGITALI

Michele De Fusco, Luigi Lo Bianco, Stefano Scarcelli, Costanza Ferrari

Relazione del progetto

Indice

1	Introduzione	2
2	Il moltiplicatore	4
2.1	Implementazione	4
2.1.1	Alcune note sui prodotti	7
2.2	Il testing	7
3	BufferLine	9
3.1	Buffer e Pipeline: L'architettura Sliding Window	9
3.1.1	Architettura del Dataflow	9
3.1.2	Implementazione degli Shift Register Arrays	10
3.1.3	Gestione del Flush e Padding	11
4	FSM ed AXI-Stream	12
4.1	Implementazione della FSM e Interfacce AXI-Stream	12
4.1.1	Architettura della FSM	12
4.1.2	Gestione del Protocollo AXI4-Stream	13
4.1.3	Sincronizzazione e Rimozione dello stato IDLE	14
4.1.4	Controllo del Dataflow e Backpressure	15

Capitolo 1

Introduzione

La traccia richiedeva l'implementazione di un circuito per il filtraggio di un'immagine greyscale di almeno 32x32 pixel rappresentati su 8 bit unsigned.

Il filtro deve essere composto da 3x3 coefficienti (scelti in modo arbitrario dal gruppo di lavoro) rappresentati su 4 bit in complemento a 2.

La finestra di convoluzione ha la stessa dimensione del filtro, quindi 3x3 e scorre lungo tutta l'immagine per filtrare un pixel alla volta; il procedimento di invio dei dati e riempimento del buffer line è gestito dalla state machine attraverso alcuni segnali che verranno descritti successivamente.

L'operazione di filtraggio deve essere condotta utilizzando un anchor point centrale quindi il pixel che subisce l'operazione di filtraggio è quello che si trova al centro della finestra.

L'immagine viene gestita attraverso il Buffer Line che restituisce i pixel di 3 righe allineati, pronti per essere inviati al moltiplicatore.

Viene rivolta particolare attenzione per i pixel che si trovano lungo i bordi e che nella finestra di convoluzione subiscono zero padding, che viene garantito dall'inizializzazione a 0 del buffer e dei vettori del componente.

Il filtraggio vero e proprio avviene attraverso l'operazione di convoluzione: i pixel dell'immagine vengono moltiplicati con il coefficiente corrispondente del filtro e infine tutti i prodotti parziali vengono sommati insieme. Il risultato sarà il pixel centrale filtrato con il Filtro Gaussiano o Gaussian Blur che ha come coefficienti:

```
2  1  2
2  4  2
1  2  1
```

Per il testing abbiamo optato per diverse immagini, tra cui alcune figure semplici che fanno risaltare molto l'influenza del filtro.

Per processare le immagini sono stati scritti diversi script MATLAB tra cui

- uno per creare semplici immagini da impiegare nel circuito;
- uno per ridimensionare immagini (prese da internet) nel formato 32x32 pixel;
- uno per estrarre le componenti numeriche di ogni immagine che saranno poi pronte per essere convertite in bit e inserite come input nel circuito;
- uno per rielaborare le componenti numeriche e ricostruire l'immagine filtrata.

Capitolo 2

Il moltiplicatore

2.1 Implementazione

La differenza principale con il moltiplicatore di Booth è che non si va a suddividere in gruppi di bit il moltiplicatore, ma si vanno a codificare tutti i possibili casi che possono accadere.

La nostra implementazione va a sfruttare i $2^4 = 16$ casi possibili per ogni coefficiente del filtro $F_{x,y}$.

La buffer line scorre sull'immagine e invia i segnali al moltiplicatore: i coefficienti del filtro sono sempre gli stessi, che poi vengono codificati nel medesimo modo, e i pixel dell'immagine vengono inviati 9 alla volta secondo una finestra 3x3.

Di seguito la entity:

```
1  entity booth_multiplier is
2      generic(
3          componente_immagine : POSITIVE := 8;
4          coefficiente_filtro : POSITIVE := 4;
5          somma : POSITIVE := 12
6      );
7      port (
8          P_1_1, P_1_2, P_1_3 : in std_logic_vector(
9              componente_immagine-1 downto 0);
10         P_2_1, P_2_2, P_2_3 : in std_logic_vector(
11             componente_immagine-1 downto 0);
12         P_3_1, P_3_2, P_3_3 : in std_logic_vector(
13             componente_immagine-1 downto 0);
14
15         F_1_1, F_1_2, F_1_3 : in std_logic_vector(
```

```

16         coefficiente_filtro-1 downto 0);
17     F_2_1, F_2_2, F_2_3 : in std_logic_vector(
18         coefficiente_filtro-1 downto 0);
19     F_3_1, F_3_2, F_3_3 : in std_logic_vector(
20         coefficiente_filtro-1 downto 0);
21
22     M_1_1, M_1_2, M_1_3 : out std_logic_vector(
23         somma-1 downto 0);
24     M_2_1, M_2_2, M_2_3 : out std_logic_vector(
25         somma-1 downto 0);
26     M_3_1, M_3_2, M_3_3 : out std_logic_vector(
27         somma-1 downto 0)
28 );
29 end entity booth_multiplier;

```

Listing 2.1: Entity del moltiplicatore

Le entrate si suddividono in questo modo:

- i segnali $P_{x,y}$ sono i pixel dell'immagine che hanno dimensione 8 bit
- i segnali $F_{x,y}$ sono i coefficienti del filtro con dimensione 4 bit

Se il moltiplicando e il moltiplicatore sono composti rispettivamente da m ed n bit il risultato del prodotto andrà espresso in $m + n$ bit.

Le uscite sono invece i segnali $M_{x,y}$ con dimensione $m + n = 8 + 4 = 12$ bit.

Il moltiplicatore segue questo funzionamento:

- L'operazione principale che abbiamo effettuato è la codifica dei 16 possibili valori che può assumere ogni coefficiente del filtro;
- Si codificano i 2 prodotti parziali $P \cdot P_A_{x,y}$ e $P \cdot P_B_{x,y}$ per ogni pixel e relativo coefficiente. Dato l'elevato numero dei casi possibili diventa complesso offrire una codifica diretta quindi si utilizzano A e B che vengono ciascuno moltiplicati per $P_{x,y}$ separatamente ottenendo i segnali $P \cdot P_A_{x,y}$ e $P \cdot P_B_{x,y}$ secondo la seguente tabella:

Case filtro	Numero da codificare	A	B	P_P_A_x_y	P_P_B_x_y
0000	0	0	0	000000000000	000000000000
0001	1	1	0	0000 P_x_y	000000000000
0010	2	-2	4	N_P_x_y(8) N_P_x_y(8) N_P_x_y 0	00 P_x_y 00
0011	3	-1	4	N_P_x_y(8) N_P_x_y(8) N_P_x_y(8) N_P_x_y	00 P_x_y 00
0100	4	0	4	000000000000	00 P_x_y 00
0101	5	1	4	0000 P_x_y	00 P_x_y 00
0110	6	-2	8	N_P_x_y(8) N_P_x_y(8) N_P_x_y 0	0 P_x_y 000
0111	7	-1	8	N_P_x_y(8) N_P_x_y(8) N_P_x_y(8) N_P_x_y	0 P_x_y 000
1000	-8	0	-8	000000000000	N_P_x_y 000
1001	-7	1	-8	0000 P_x_y	N_P_x_y 000
1010	-6	-2	-4	N_P_x_y(8) N_P_x_y(8) N_P_x_y 0	N_P_x_y(8) N_P_x_y 00
1011	-5	-1	-4	N_P_x_y(8) N_P_x_y(8) N_P_x_y(8) N_P_x_y	N_P_x_y(8) N_P_x_y 00
1100	-4	0	-4	000000000000	N_P_x_y(8) N_P_x_y 00
1101	-3	1	-4	0000 P_x_y	N_P_x_y(8) N_P_x_y 00
1110	-2	-2	0	N_P_x_y(8) N_P_x_y(8) N_P_x_y 0	000000000000
1111	-1	-1	0	N_P_x_y(8) N_P_x_y(8) N_P_x_y(8) N_P_x_y	000000000000

Da notare che $N_P_x_y$ ha dimensione 9 bit ed esprime la negazione del pixel P_x_y , quindi corrisponde alla sua inversione dei singoli bit e alla somma di '1' attraverso una batteria di Ripple Carry Adder, uno per ogni pixel e si utilizza quando i prodotti parziali $P_P_A_x_y$ oppure $P_P_B_x_y$ sono negativi.

Da ricordare che $P_P_A_x_y$ e $P_P_B_x_y$ devono essere di 12 bit quindi vanno gestiti bene i prodotti da effettuare attraverso gli shift sinistri e l'estensione che deve portare ogni segnale alla dimensione giusta.

Questa operazione all'interno del codice risulta in 9 case statement in cui si codificano le componenti $P_P_A_x_y$ e $P_P_B_x_y$ per tutti i coefficienti del filtro.

- L'ultimo passaggio è la somma fra le componenti $P_P_A_x_y$ e $P_P_B_x_y$ calcolata per tutti gli ingressi attraverso dei semplici circuiti sommatore Ripple Carry.

2.1.1 Alcune note sui prodotti

I prodotti vengono effettuati in modo intelligente, provando a sfruttare operazioni immediate come shift sinistri e negazioni, quindi quelli per 0, +1, -1, +2, -2, +4, -4 e così via, nello stesso modo in cui si calcolano nel moltiplicatore di Booth.

Questi prodotti sono semplici da ottenere:

- Prodotto per 0 => Il prodotto parziale avrà tutti i bit a 0;
 - Prodotto per 1 => Il prodotto parziale sarà uguale al moltiplicando;
 - Prodotto per -1 => Il prodotto parziale sarà uguale al moltiplicando che subisce l'inversione dei bit e a cui viene sommato 1;
 - Prodotto per 2 => Il prodotto parziale sarà uguale al moltiplicando che subisce uno shift sinistro;
 - Prodotto per -2 => Il prodotto parziale sarà uguale al moltiplicando che subisce l'inversione dei bit, la somma di 1 e uno shift sinistro;
 - Prodotto per 4 => Il prodotto parziale sarà uguale al moltiplicando che subisce due shift sinistri;
 - Prodotto per -4 => Il prodotto parziale sarà uguale al moltiplicando che subisce l'inversione dei bit, la somma di 1 e due shift sinistri;
- e così via.

La strategia è quella di utilizzare A e B per ottenere i prodotti non immediati (ad esempio 3, 5, 6, 7 e così via) combinando i prodotti parziali $P.P.A_{x,y}$ e $P.P.B_{x,y}$ attraverso la somma finale.

Ad esempio nel caso in cui il coefficiente abbia valore 5 (quindi 0101) si ricorre a codificare il prodotto per A=1 e per B=4, si determinano i parziali da sommare insieme e si avrà ottenuto il pixel filtrato.

2.2 Il testing

Si allegano ora i risultati del testbench effettuato in cui figurano i risultati correttamente calcolati:

> ♡ P_1_1[7:0]	255	0	255	0	255							
> ♡ P_1_2[7:0]	255	0	255	0	255							
> ♡ P_1_3[7:0]	255	0	255	0	255							
> ♡ P_2_1[7:0]	255	0	255	0	255							
> ♡ P_2_2[7:0]	255	0	255	0	255							
> ♡ P_2_3[7:0]	255	0	255	0	255							
> ♡ P_3_1[7:0]	255	0	255	0	255							
> ♡ P_3_2[7:0]	255	0	255	0	255							
> ♡ P_3_3[7:0]	255	0	255	0	255							
> ♡ F_1_1[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2		
> ♡ F_1_2[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2		
> ♡ F_1_3[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2		
> ♡ F_2_1[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2		
> ♡ F_2_2[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2		
> ♡ F_2_3[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2		
> ♡ F_3_1[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2		
> ♡ F_3_2[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2		
> ♡ F_3_3[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2		
> ♡ M_1_1[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510
> ♡ M_1_2[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510
> ♡ M_1_3[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510
> ♡ M_2_1[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510
> ♡ M_2_2[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510
> ♡ M_2_3[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510
> ♡ M_3_1[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510
> ♡ M_3_2[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510
> ♡ M_3_3[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510

Capitolo 3

BufferLine

3.1 Buffer e Pipeline: L'architettura Sliding Window

La trasformazione di un immagine in un contesto spaziale bidimensionale è il compito principale del modulo `buffer_line`. Questo blocco è responsabile della creazione della "finestra scorrevole" (Sliding Window) 3×3 necessaria per l'operazione di convoluzione.

3.1.1 Architettura del Dataflow

Il modulo implementa una struttura a registri a scorrimento (Shift Registers) interconnessi con buffer di linea (FIFO). L'obiettivo è rendere disponibili contemporaneamente, in un singolo ciclo di clock, i 9 pixel adiacenti necessari al filtro:

""Immagine da inserire""

$$\text{Window} = \begin{bmatrix} d_{00} & d_{01} & d_{02} \\ d_{10} & d_{11} & d_{12} \\ d_{20} & d_{21} & d_{22} \end{bmatrix}$$

I dati in ingresso dall'interfaccia AXI-Stream vengono inseriti nel registro d_{00} . Ad ogni colpo di clock, se l'abilitazione della pipeline è attiva, i dati scorrono verso destra. Al termine della prima riga della finestra (d_{02}), il dato entra in un *Line Buffer* che lo ritarda di un numero di cicli pari alla larghezza rimanente dell'immagine, per poi riapparire all'inizio della seconda riga (d_{10}).

Questa architettura garantisce che, una volta riempita la pipeline (latenza iniziale), i registri contengano sempre pixel spazialmente adiacenti, nonostante arrivino temporalmente sequenziali.

3.1.2 Implementazione degli Shift Register Arrays

Le FIFO necessarie per memorizzare le righe dell'immagine sono realizzate utilizzando array parametrici di vettori logici. La dimensione di questi buffer è critica e dipende dalla larghezza dell'immagine (N_{col}) e dalla dimensione del kernel ($K_{dim} = 3$).

La profondità del buffer è calcolata come:

$$\text{Buffer Depth} = N_{col} - K_{dim}$$

Nel codice VHDL (Listing 3.1), viene definito un tipo array per istanziare i due buffer di linea necessari per una finestra 3×3 .

```

1  -- Definizione dei tipi per il buffer
2  -- Esempio: Image Width (32) - Window Size (3) = 29
   elementi di buffer
3  -- La dimensione e' parametrica basata su ncol
4  type reg_array is array (ncol-4 downto 0) of
   std_logic_vector(7 downto 0);
5  signal buffer1, buffer2 : reg_array;
6
7  -- Processo di scorrimento (Line Buffer 1)
8  process(clk)
9  begin
10     if rising_edge(clk) then
11         if pipeline_en = '1' then
12             -- Ingresso dal registro finale della riga precedente
               (d02)
13             buffer1(0) <= d02_s;
14             -- Shift dell'intera riga nel buffer
15             for j in 1 to ncol-4 loop
16                 buffer1(j) <= buffer1(j-1);
17             end loop;
18
19             -- L'uscita del buffer alimenta la riga successiva (d10
               )
20             d10_s <= buffer1(ncol-4);
21         end if;
22     end if;

```

```
23 end process;
```

Listing 3.1: Definizione e processo di shift dei Line Buffer

3.1.3 Gestione del Flush e Padding

Una criticità nei filtri digitali è la gestione dei bordi dell'immagine. Quando l'ultimo pixel valido entra nella pipeline, la finestra di elaborazione non ha ancora terminato il suo lavoro: l'ultimo pixel deve infatti scorrere fino a raggiungere la posizione centrale (d_{11}) affinché l'operazione di filtraggio sia completa per tutto il frame.

Per gestire questa fase, il sistema implementa una logica di *Zero Padding* attivata dallo stato di **FLUSH** della FSM.

Logica di Inserimento Zeri

Quando il segnale `flush` è attivo, l'ingresso fisico della pipeline viene disaccoppiato dall'interfaccia AXI e forzato a zero. Questo permette di "spingere" i dati validi residui attraverso i registri e i buffer di linea senza introdurre nuovi valori che corromperebbero il calcolo.

```
1  if rising_edge(clk) and pipeline_en = '1' then
2  -- Se siamo in fase di FLUSH, carichiamo zeri (Padding)
3  -- per svuotare la pipeline dai pixel validi residui
4  if flush_state = '1' then
5      d00_s <= (others => '0');
6  else
7      -- Durante il normale funzionamento, acquisiamo il dato
      AXI
8      d00_s <= s_axis_tdata;
9  end if;
10
11 -- Shift dei registri di finestra (Row 0)
12 d01_s <= d00_s;
13 d02_s <= d01_s;
14 end if;
```

Listing 3.2: Multiplexer di ingresso per gestione Flush e Padding

Questa tecnica assicura che l'immagine in uscita abbia esattamente le stesse dimensioni di quella in ingresso, risolvendo il problema della latenza intrinseca della pipeline.

Capitolo 4

FSM ed AXI-Stream

4.1 Implementazione della FSM e Interfacce AXI-Stream

Il cuore del sistema di filtraggio è costituito dalla logica di controllo, implementata attraverso una Macchina a Stati Finiti (FSM) che orchestra il riempimento dei *buffer line*, la generazione della finestra 3×3 e la gestione dei segnali del protocollo AXI4-Stream. In questo capitolo vengono analizzati nel dettaglio il diagramma degli stati, la gestione dei segnali di sincronizzazione (SOF ed EOL) e la logica di controllo del flusso dati (*backpressure*).

4.1.1 Architettura della FSM

La FSM ha il compito di coordinare tre segnali di controllo fondamentali per il funzionamento della pipeline:

- **pipeline_en**: segnale di abilitazione globale che determina l'avanzamento dei dati nei registri a scorrimento;
- **window_valid**: flag che indica se la finestra di convoluzione attuale contiene dati validi per il calcolo;
- **flush_pipeline**: segnale che gestisce la fase terminale dell'elaborazione, forzando l'ingresso a zero quando non vi sono più pixel in arrivo ma la pipeline deve ancora svuotarsi.

A differenza di implementazioni classiche, lo stato di **IDLE** è stato rimosso dalla logica esplicita degli stati per ottimizzare la risposta al segnale di *Start Of Frame* (SOF). La macchina opera principalmente su tre stati:

1. **LOADING**: In questa fase iniziale, il sistema riceve i dati in ingresso e riempie i *line buffer*. La finestra di convoluzione non è ancora valida e l'uscita è disabilitata.
2. **RUNNING**: Una volta riempita la finestra, il sistema entra in regime di elaborazione continua. Per ogni colpo di clock (se abilitato), viene prodotto un pixel filtrato in uscita.
3. **FLUSH**: Alla ricezione dell'ultimo pixel dell'immagine, la macchina entra in fase di svuotamento. L'ingresso della pipeline viene alimentato con valori nulli (zero padding) per permettere l'elaborazione dei pixel ai bordi finali dell'immagine. Completato lo svuotamento, la FSM torna in attesa.

4.1.2 Gestione del Protocollo AXI4-Stream

Il modulo agisce come un nodo di elaborazione trasparente all'interno della catena video, comportandosi come *Slave* sull'interfaccia di ingresso e come *Master* su quella di uscita.

Interfaccia Slave (Ingresso)

L'accettazione dei dati è governata dal segnale `s_axis_tready`. Questo viene asserito alto, segnalando la disponibilità a ricevere dati, a condizione che la FSM non si trovi nello stato di **FLUSH** e che il modulo a valle non stia esercitando *backpressure*, ovvero la porta in uscita non è pronta a ricevere dati. I segnali di controllo del frame vengono gestiti come segue:

- **SOF (Start Of Frame - tuser)**: Viene campionato per resettare istantaneamente la FSM e i contatori interni, garantendo l'allineamento all'inizio di ogni nuova immagine.
- **EOL (End Of Line - tlast)**: Utilizzato per il conteggio delle righe e per innescare la transizione verso lo stato di **FLUSH** al termine del caricamento dell'ultimo pixel.

Interfaccia Master (Uscita)

In uscita, il segnale di validità `m_axis_tvalid` riflette lo stato interno del segnale `window_valid`.

- Il segnale `m_axis_tuser` viene asserito per un singolo ciclo di clock in corrispondenza del primo pixel valido uscente.
- Il segnale `m_axis_tlast` viene generato internamente da un contatore di colonne che traccia la posizione del pixel corrente all'interno della riga elaborata.

4.1.3 Sincronizzazione e Rimozione dello stato IDLE

Per massimizzare l'efficienza e la reattività, la logica di reset e avvio è basata sul segnale `s_axis_tuser`. Invece di attendere in uno stato inattivo, la FSM monitora costantemente l'arrivo di un nuovo frame. Come mostrato nel Listing 4.1, l'arrivo di un `tuser` alto forza la macchina nello stato di `LOADING` e azzeri i contatori di latenza, indipendentemente dallo stato attuale.

```
1  -- Processo Sincrono della FSM
2  if rising_edge(s_axis_clk) then
3      if s_axis_rstn = '0' then
4          current_state <= LOADING;
5
6      elsif pipeline_en_s = '1' then
7          -- LOGICA DI SINCRONIZZAZIONE SOF
8          -- Un TUSER valido indica SEMPRE l'inizio di un nuovo
9          frame
10         if s_axis_tvalid = '1' and s_axis_tuser = '1' then
11             current_state <= LOADING;
12             latency_counter <= 1; -- Inizializzazione contatore
13             flush_pipeline_s <= '0'; -- Interruzione flush
14             precedenti
15             m_axis_tuser_s <= '0';
16         else
17             -- Evoluzione normale degli stati (LOADING, RUNNING,
18             FLUSH)
19             -- ...
20         end if;
21     end if;
22 end if;
```

Listing 4.1: Gestione sincrona del SOF e reset implicito

4.1.4 Controllo del Dataflow e Backpressure

La robustezza del filtro digitale in un contesto AXI-Stream dipende dalla corretta gestione della *backpressure*. Il sistema deve essere in grado di "congelare" l'intera pipeline se il ricevitore a valle non è pronto ad accettare dati (`m_axis_tready = '0'`).

La logica combinatoria che genera il segnale di abilitazione `pipeline_en_s` è descritta dalla seguente espressione VHDL:

```
1  pipeline_en_s <= '1' when ((s_axis_tvalid = '1' or
2      current_state = FLUSH)
3      and (m_axis_tready = '1' or window_valid_s = '0'))
4      else '0';
5
6  s_axis_tready <= '1' when (current_state /= FLUSH and
    m_axis_tready = '1')
    else '0';
```

Listing 4.2: Logica di abilitazione pipeline e gestione backpressure

Analizzando il codice nel Listing 4.2, l'abilitazione avviene solo se:

1. Vi è un dato valido in ingresso o la macchina è in fase di svuotamento (sorgente dati interna);
2. **E** il ricevitore è pronto (`m_axis_tready = '1'`) o l'uscita attuale non è valida (quindi non c'è rischio di perdere dati).

Se `m_axis_tready` scende a livello logico basso mentre è presente un dato valido in uscita, il segnale `pipeline_en_s` viene deassertito. Questo causa l'arresto immediato di tutti i registri a scorrimento e dei buffer, preservando lo stato interno del filtro fino a quando la congestione a valle non viene risolta.